What is kain difference between Library and framework if both have predefined functions and already done work

Good question — and your confusion is very common. Yes, both libraries and frameworks provide pre-written code, but the *real difference is who controls the flow of the program.*

🔑 Core Difference (Simple Idea)

- Library → You are in control
- Framework → It is in control

📚 Library (You call it)

A library is a collection of functions that you use whenever you want.

👉 You decide:

- when to call it
- where to use it
- how to use it

Example:

If you're using a math library:

```
sqrt(16);
```

You called it — it didn't call you.

💡 Think like:

> "I need help → I call the library"

🏗️ Framework (It calls you)

A framework already defines the structure of your program.

👉 It decides:

- when your code runs
- where your code fits

You just fill in the missing parts.

Example:

In a web framework:

```
def handle_request():
    return "Hello"
```

You didn't call this — the framework calls it when a request comes.

💡 Think like:

> "Framework is boss → it calls me when needed"

⚡ Key Concept: Inversion of Control (IoC)

This is the real difference:

- Library → Normal control
- Framework → Inversion of Control (IoC)

Meaning:
👉 Control goes from *you* → *framework*

🎯 Easy Analogy

- Library = Ordering food
  → You choose what to order
- Framework = Fixed menu restaurant
  → System is already set, you just pick options

🔍 Quick Comparison

Feature	Library	Framework
Control	You control	Framework controls
Usage	Call functions	Follow structure
Flexibility	High	Less (structured)
Example	C++ STL, jQuery	Django, Angular

💡 Final Simple Line

👉 Library = Tool you use

👉 Framework = System you work inside

If you want, I can ↪ give examples specifically in C++ (without STL) or ↪ relate this to your assignments — that'll make it even clearer.

Explain in more easy and simple way along with proper example

Alright — let's make this *very simple and crystal clear*.

🎯 One-Line Difference

👉 Library = You call it

👉 Framework = It calls you

📚 Library (Easy Example)

Imagine you are writing a program and you need to calculate square root.

You use a library function:

```
sqrt(25);
```

👉 What happened here?

- You wrote the main program
- You decided when to use `sqrt`
- Library just helped you

💡 So:

> You are the boss, library is your helper

🧠 Real Life Analogy

Library = Calculator

- You pick it up when needed
- You press buttons
- It never tells you what to do

🏗️ Framework (Easy Example)

Now imagine you are making a website using a framework like

Django

You write:

```
def home(request):
    return "Hello"
```

👉 But you never call this function!

- When user opens website → Django calls your function
- You just wrote the logic
- Framework controls everything

💡 So:

Framework is the boss, you follow its system

🧠 Real Life Analogy

Framework = School System

- Timetable already made
- Teacher decides when class starts
- You just go and study

⚡ Side-by-Side Super Simple

Situation	Library	Framework
Who is boss?	You	Framework
Who calls function?	You call	It calls
Control	In your hands	Pre-defined

🔥 Final Example (Clear Difference)

Library style:

```
int main() {
    printHello();  // YOU called it
}
```

Framework style:

```
def printHello():
    return "Hello"
```

👉 Somewhere inside framework:

```
# Framework does this (you don't see it)
printHello()
```

🧠 Final Memory Trick

👉 Library = "I call"

👉 Framework = "Call me when needed"

If you want, I can give a C++-only example without STL or a real-world programming scenario (like your assignment) to make it even stronger.